Lab. With that in mind, the target audience for the GDK is
anyone who is contributing to the GitLab project. This includes:

- Software engineers directly writing code
- Software engineers in test who are testing that code
- Product designers verifying designs are implemented correctly
- UX researchers demonstrating functionality to users for research purposes
- Product managers and engineering managers performing acceptance testing
  before code is merged to master
- All of the above, whether they are working for GitLab or are part of
  our broader community of contributors.

GitLab is an Open Core product,
and our community is central to the continued success of our mission. This
means that tools that allow anyone to jump in and contribute are vital to
keeping that momentum going.

Future Vision

We want to build towards a state in which any GitLab contributor can easily install and use the GDK
with minimal effort. By "lowering the bar" to getting started, we allow our contributors to focus
on designing and building our next great feature.

We're iterating toward a version of the GDK that can install in a single command, updates itself
painlessly, and allows contributors to check out a feature branch with minimal effort.

Challenges to address

Getting started with the GDK is straightforward for GitLab employees because
of the tribal knowledge and in-company training we do. We need to move that
knowledge in to the documentation and the handbook to make it just as easy for
anyone in the community to get ramped up.

Examining a specific branch is a central workflow for many of our personas.
These users would benefit greatly from a minimal-fuss, no-configuration way to
switch to a specific branch to verify various aspects of the application with
high velocity.

Making GDK updates a smooth experience is a high-priority issue, as there
are a large number of reports of breaks happening when upgrading. This makes it
painful to upgrade and smoothly continue working, or (even worse) discourages
developers from upgrading in a timely manner.

Getting help with the GDK isn't so bad for GitLab Team Members, since they
can just join the gdk (internal) Slack channel. This
approach is not suitable for community members, and we need a better solution
that works well for everyone (so that everyone can contribute.)

What's Next & Why

Accelerate gdk install/update/reconfigure commands
by optimizing logic to only perform necessary actions.

Support installing asdf binary packages
which will greatly improve the speed and reliability when running gdk install/update.

Improve Gitpod startup time
by optimizing Docker image size, creation workflow and performing less logic
on startup.

Improve the GDK Documentation
to increase clarity particularly for new contributors. This will include
reorganizing content to make it more discoverable and scannable, improving our
"Preparing your computer" recommendations, and updating the documentation
for the latest requirements and prerequisites.

What we are not doing

Integrating the GitLab Compose Kit (GCK). The GCK project
is also building local development environment tooling, however, the architecture
of the two approaches are fundamentally incompatible. See this note on that project's readme
for more information.

Why is this important?

The GDK is key to our Strategy,
as the GDK is the entrypoint to contribution to the GitLab project itself. Without
the GDK, many contributors would have to learn the specifics of every
service that a local environment must run to function, the commands required to
run each of those services, and the order they must be stood up in to function
properly. They'd have to run these commands manually, or more likely, write some
tooling that automates the process for them... which after significant amounts
of time and effort were put in to those scripts, it would probably look a lot like the GDK.

SECTION: engineering/infrastructure/engineering-productivity/issue-triage.md

GitLab believes in Open Development, and we encourage the community to file issues and open merge requests for our projects on GitLab.com. Their contributions are valuable, and we should handle them as effectively as possible. A central part of this is triage - the process of categorization according to type and severity.

Any GitLab team-member can triage issues. Keeping the number of un-triaged issues low is essential for maintainability, and is our collective responsibility. Consider triaging a few issues around your other responsibilities, or scheduling some time for it on a regular basis.

Partial Triage

The Engineering Productivity team own the issue triage process, but there is no capacity to manually triage issues without a group label at present.
We rely on a combination of self and AI triage.

Partial Triage checklist

- Issue is spam:
  - Report the issue.
  - Make the issue confidential.
  - Post a link to the issue in the abuse slack channel.
- Issue is request for help:
  - Post the support message and close the issue.
  - Alternatively, ask for more information and apply the ~"awaiting feedback" label.
- Issue is duplicate:
  - Post the duplicate message.
  - Call the /duplicate action to create the link to the original and close the issue.
- Assign a type label.
  - If it is unclear whether the issue is a bug or a support request, tag the PM/EM for the group and ask for their opinion.
- ~"type::bug": assign a severity label.
  - If ~"severity::1" or ~"severity::2": mention the PM/EM from the group
- Assign a group label.
  - If there is no suitable group label: assign a stage ("devops") label.
- Optionally tag relevant domain experts.

Complete Triage

An issue is considered completely triaged when all of the following criteria are met:

- It is partially triaged.
- It has a milestone set.
- It has a priority label applied for ~"type::bug" and ~"Deferred UX".

Type Labels

Type labels are defined on the Engineering Metrics page.
If you are unsure about the type, you can tag the product or engineering manager for the group and ask their opinion.

Group labels

Assigning a group label allows gitlab-bot to automatically assign the right stage label.
The Features by Group listing can help find the right group.

Priority

The priority label is used to indicate the importance and guide the scheduling of the issue. Priority labels are expected to be set based on the circumstances of the market, product direction, IACV impact, number of impacted users and capacity of the team. DRIs for prioritization are based on work type:

- Feature - Product Manager (PM)
- Maintenance - Engineering Manager (EM)
- Bug - Quality Engineering Manager (QEM)

Priority	Importance	Intention	DRI
~"priority::1"	Urgent	We will address this as soon as possible regardless of the limit on our team capacity. Our target resolution time is 30 days.	PM, EM, or QEM of that product group, based on work type
~"priority::2"	High	We will address this soon and will provide capacity from our team for it in the next few releases. This will likely get resolved in 60-90 days.	PM, EM, or QEM of that product group, based on work type
~"priority::3"	Medium	We want to address this but may have other higher priority items. This will likely get resolved in 90-120 days.	PM, EM, or QEM of that product group, based on work type
~"priority::4"	Low	We don't have visibility when this will be addressed. No timeline designated.	PM, EM, or QEM of that product group, based on work type

Severity

If you need help estimating severity, reach out in the sdeveloperexperience channel.

Note: These severity definitions apply to issues only. Please see Severity Levels section of the Incident Management page for details on incident severity.

Severity labels help us determine urgency and clearly communicate the impact of a ~"type::bug" on users. There can be multiple categories of a ~"type::bug".

The presence of bug category labels ~"bug::availability", ~"bug::performance", ~"bug::vulnerability", and ~UX denotes to use the severity definition in that category. When a ~"type::bug" correspond to multiple categories, the severity to apply should be the higher, for example, if an issue has a ~"severity::2" for ~"bug::availability" and a ~"severity::1" for ~"bug::performance" then the severity assigned to the issue should be ~"severity::1".

Once you've determined a severity for an issue add a note that explains in summary why you selected the severity you did. This will help future team members understand your rationale so they will know how to proceed with acting upon the issue.

Type of ~"type::bug"	~"severity::1": Blocker	~"severity::2": Critical	~"severity::3": Major	~"severity::4": Low	Triage DRI

---|---|---|---|---|
General bugs	Broken feature with no workaround or any data-loss.	Broken feature with an unacceptably complex workaround.	Broken feature with a workaround.	Functionality is inconvenient.
SET or EM for that product group.				
~"bug::performance" Response time (API/Web/Git)^1]	Above 9000ms to timing out	Between 2000ms and 9000ms	Between 1000ms and 2000ms	Between 200ms and 1000ms
[Performance Enablement group				
~"bug::performance" Browser Rendering (LCP)^2]	Above 9000ms to timing out	Between 4000ms and 9000ms	Between 3000ms and 4000ms	Between 3000ms and 2500ms
[Performance Enablement group				
~"bug::performance" Browser Rendering (TBT)^2]	Above 9000ms to timing out	Between 2000ms and 9000ms	Between 1000ms and 2000ms	Between 300ms and 1000ms
[Performance Enablement group				
~bug::ux User experience problem ³	"I can't figure this out." Users are blocked and/or likely to make risky errors due to poor usability, and are likely to ask for support.	"I can figure out why this is happening, but it's really painful to solve." Users are significantly delayed by the available workaround.	"This still works, but I have to make small changes to my process." Users are self sufficient in completing the task with the workaround, but may be somewhat delayed.	"There is a small inconvenience or inconsistency." Usability isn't ideal or there is a small cosmetic issue.
Product Designers of that Product group				
~"bug::availability" of GitLab SaaS	See Availability section	See Availability section	See Availability section	See Availability section
~"bug::vulnerability" Security Vulnerability	See Vulnerability Remediation SLAs	See Vulnerability Remediation SLAs	See Vulnerability Remediation SLAs	See Vulnerability Remediation SLAs
AppSec team				
Global Search	See Search Prioritization	See Search Prioritization	See Search Prioritization	

See Search Prioritization |
|
| ~test Bugs blocking end-to-end test execution | See Blocked tests section
| See Blocked tests section
| See Blocked tests section
| See Blocked tests section
| Developer Experience stage |
| ~GitLab.com Resource Saturation Capacity planning warnings | Mean forecast shows Hard SLO breach within 3 months.
|
|
|
| Scalability Engineering Manager (who will hand over to EM that owns the resource)
|

Severity SLOs

The severity label also helps us define a completion time for issues with the following labels:

- ~"type::bug"
- ~"infradev"

This indicates the expected timeline & urgency which is used to measure our SLO targets.

Severity	~infradev SLO	~"type::bug" resolution SLO	~"GitLab.com Resource Saturation" resolution SLO	Security ~vulnerability SLO
~"severity::1"	1 week	The current release + next available deployment to GitLab.com (within 30 days)	Within 2 months	See Vulnerability Remediation SLAs
~"severity::2"	30 days	The next release (60 days)		See Vulnerability Remediation SLAs
~"severity::3"	60 days	Within the next 3 releases (approx one quarter or 90 days)		See Vulnerability Remediation SLAs
~"severity::4"	90 days	Anything outside the next 3 releases (more than one quarter or 120 days).		See Vulnerability Remediation SLAs

Examples of severity levels

If a issue seems to fall between two severity labels, assign it to the higher severity label.

- Example(s) of ~"severity::1"
  - Data corruption/loss.
  - Security breach.
  - Unable to create an issue or merge request.
  - Unable to add a comment or thread to the issue or merge request.
  - An error message displays (that looks like a blocker) when the message should instead be informational.
  - Unclear instructions in the UI that lead to irreversible changes.

- Example(s) of ~"severity::2"
  - Cannot submit changes through the web IDE, but the command line works.
  - A status widget on the merge request page is not working, but information can be seen in the test pipeline page.
  - A workaround is available but it requires the use of the Rails console, making it unacceptably complex.
- Example(s) of ~"severity::3"
  - Can create merge requests only from the Merge Requests list view, not from an Issue page.
- Example(s) of ~"severity::4"
  - Incorrect colors.
  - Misalignment.

Re-evaluating closed issues

As the triager of an issue you are responsible for adjusting your decision based on additional information that surfaces later.
To do that, track subsequent activity on issues that you have closed and adjust your decision as needed.

Availability

Issues with ~"bug::availability" label directly impacts the availability of GitLab.com SaaS. It is considered as another category of ~"type::bug".

For the purposes of Incident Management, incident issue severities are chosen based on the availability severity matrix below.

We categorize these issues based on the impact to GitLab.com's customer business goal and day to day workflow.

The prioritization scheme adheres to our product prioritization where security and availability work are prioritized over feature velocity.

The presence of these severity labels modifies the standard severity labels(~"severity::1", ~"severity::2", ~"severity::3", ~"severity::4") by primarily taking into account the impact to users. The severity of these issues may change depending on the re-analysis of the impact to GitLab.com users.

Severity	Availability impact	Time to mitigate (TTM)(1)	Time to resolve (TTR)(2)	Minimum priority
~"severity::1"	Problem on GitLab.com blocking the typical user's workflow Impacts 20% or more of users without an available workaround AND/OR Any roadblock that puts the guaranteed self-managed release date at risk (use ~backstage label) AND/OR Any data loss directly impacting customers	Within 8 hrs	Within 48 hrs	~"priority::1"
~"severity::2"	Problem on GitLab.com blocking the typical user's workflow Impacts 20% or more of users, but a reasonable workaround is available. Impacts between 5%-20% of users without an available workaround	Within 24 hrs	Within 7 days	~"priority::1"
~"severity::3"	Broad impact on GitLab.com and minor inconvenience to typical user's

workflow. No workaround needed.

Impacts up to 5% of users | Within 72 hrs | Within 30 days | ~"priority::2" |
| ~"severity::4" | Minimal impact on GitLab.com typical user's workflow to less than 5% of users

May also include incidents with no impact, but with importance to resolve to prevent future risk| Within 7 days | Within 60 days | ~"priority::3" |

(1) - Mitigation uses non-standard work processes, eg. hot-patching, critical code and configuration changes. Owned by Infrastructure department, leveraging available escalation processes (dev-escalation and similar)

(2) - Resolution uses standard work processes, eg. code review. Scheduling is owned by the Product department, within the defined SLO targets.

Availability prioritization

The priority of an availability issue is tied to severity in the following manner:

Issue with the labels	Allowed priorities	Not-allowed priorities
~"bug::availability" ~"severity::1"	~"priority::1" only	~"priority::2", ~"priority::3", and ~"priority::4"
~"bug::availability" ~"severity::2"	~"priority::1" only	~"priority::2", ~"priority::3", and ~"priority::4"
~"bug::availability" ~"severity::3"	~"priority::2" as baseline, ~"priority::1" allowed	~"priority::3", and ~"priority::4"
~"bug::availability" ~"severity::4"	~"priority::3" as baseline, ~"priority::2" and ~"priority::1" allowed	~"priority::4"

Merge requests experience

The merge request (MR) experience is the core of our product. Due to many teams contributing to the MR workflow components, it has become a disjointed experience.

The overlapping is largely seen in the following areas: Merge Request Widgets, Mergeability Checks, MWPS and Merge Trains.

As part of the analysis in the Transient Bug working group, we have discovered that the top most affected product areas are:

1. create::code review
1. verify::continuous integration
1. create::source code (tied)
1. plan::project management (tied)

These product groups also have a high sensitivity to GMAU. This product groups will benefit from a heightened awareness on bugs overlapping with Merge Request functionality.

Merge requests experience prioritization

We need an elevated sense of action in this area. If a bug is related to the merge request

experience it should have the labels ~UX ~merge requests.
Priority is tied to severity in the following manner:

MR UX bug severity	Allowed priorities	Not-allowed priorities
~"severity::1"	~"priority::1" only	~"priority::2", ~"priority::3" and ~"priority::4"
~"severity::2"	~"priority::1" only	~"priority::2", ~"priority::3" and ~"priority::4"
~"severity::3"	~"priority::1" or ~"priority::2"	~"priority::3" and ~"priority::4"
~"severity::4"	~"priority::1" or ~"priority::2" or ~"priority::3"	~"priority::4"

Blocked tests

End-to-end tests that don't run lead to blind spots that can cause unforeseen availability issues.
We must ensure coverage is stable and active by quickly resolving issues that cause quarantined end-to-end tests.

Blocked tests prioritization

To promote awareness of bugs blocking end-to-end test execution, newly opened ~test ~"type::bug" issues will be announced in several Slack channels:

- All newly opened bugs blocking end-to-end test execution should be announced in quality channel.
- A newly opened bug blocking end-to-end test execution that is a product bug should also be announced in development and vp-development channels with the appropriate EM and PM tagged.

Priority is tied to severity in the following manner:

Type of test blocked	Bug severity	Allowed priorities	Not-allowed priorities
Smoke end-to-end test	~"severity::1"	~"priority::1" only	~"priority::2", ~"priority::3" and ~"priority::4"
Non-smoke end-to-end test	~"severity::2"	~"priority::2" as baseline, ~"priority::1" allowed	~"priority::3" and ~"priority::4"

Performance

Improving performance: It may not be possible to reach the intended response time in one iteration.
We encourage performance improvements to be broken down. Improve where we can and then re-evaluate the next appropriate level of severity & priority based on the new response time.

^1]: Our current response time targets for APIs, Web Controllers and Git calls are based on the TTFB P90 results of the [GitLab Performance Tool (GPT) being run against a 200 RPS / 10k User reference architecture class environment under lab like conditions. This run happens nightly and results are outputted to the Reference Architecture project wiki. These targets are owned by the GitLab Delivery: Framework team.

^2]: Our current Browser Rendering targets for [Largest Contentful Paint (LCP) and Total

Blocking Time (TBT) are based on results of SiteSpeed being run against a 20 RPS / 1k User reference architecture class environment under lab like conditions. This run happens nightly and results are outputted to the wiki on the GBPT project. These targets are owned by the Developer Experience: Performance Enablement team.

UX

UX bugs

Some UX-related issues are known to impact our System Usability Scale (SUS) score, which is a focus in our three-year strategy. We particularily target issues with the label bug::ux. These issues will have a severity label applied and they follow the severity and SLOs for type::bug issues.

Deferred UX

Issues labeled as ~Deferred UX also have a severity (and additionally priority) label applied without an accompanying ~"type::bug" label. Deferred UX results from the decision to release a user-facing feature that needs refinement, with the intention to improve it in subsequent iterations. Because it is an intentional decision, ~Deferred UX should not have a severity higher than ~"severity::3", because MVCs should not intentionally have obvious bugs or significant usability problems. If you find yourself creating a Deferred UX issue that is higher than ~"severity::3", please talk to your stage group team about reincorporating that issue into the MVC.

Transient bugs

A transient bug is unexpected, unintended behavior that does not always occur in response to the same action.

Transient bugs give users conflicting impressions about what is happening when they take action, may not consistently occur, and last for a short period of time. While these bugs may not block a user's workflow and are usually resolved by a total page refresh, they are detrimental to the user experience and can build a lack of trust in the product. Users can become unsure about whether the data they are seeing is stale, fresh, or has even updated after they took an action. Examples of transient behaviors include:

- Clicking the "Apply Suggestion" button and the page not getting updated with the applied suggestion
- Updating the milestone of an issue by using a quick action, but the sidebar not updating to reflect the new milestone
- Merging a merge request and the merge request page still showing as "Open"

In order to define an issue as a "transient bug," use the ~"bug::transient" label

Infradev Issues

An issue may have an infradev label attached to it, which means it subscribes to a dedicated process to related to SaaS availability and reliability, as detailed in the Infradev Engineering Workflow. These issues follow the established severity SLOs for bugs.

Limit Related Bugs

GitLab, like most large applications, enforces limits within certain features. The absences of limits can impact security, performance, and availability. For this reason issues related to limits are considered ~"type::bug" in the ~"bug::availability" sub-category.

In order to define an issue as related to limits add the labels ~"availability::limit" and ~"bug::availability".

Severity should be assessed using the following table:

Severity	Availability impact
~"severity::1"	Absence of this limit enables a single user to negatively impact availablity of GitLab
~"severity::2"	Absence of this limit poses a risk to reduced availability of GitLab
~"severity::3"	Absence of this limit has a negative impact on ability to manage cost, performance, or availability
~"severity::4"	A limit could be applied, but it's absences does not pose availability risk

These issues follow the established severity SLOs for bugs.

Triaging Issues

Initial triage involves (at a minimum) labelling an issue appropriately, so un-triaged issues can be discovered by searching for issues without any labels.

Follow one of these links:

- GitLab
- GitLab Omnibus
- GitLab.com Support Tracker

Pick an issue, with preference given to the oldest in the list, and evaluate it with a critical eye, bearing the issue triage practices below in mind. Some questions to ask yourself:

- Do you understand what the issue is describing?
- What labels apply? Particularly consider type, stage and severity labels.
- How critical does it seem? Does it need to be escalated to a product or engineering manager, or to the security team?
- Would the ~"bug::vulnerability" label be appropriate?
- Should it be made confidential? It's usually the case for ~"bug::vulnerability" issues or issues that contain private information.

Apply each label that seems appropriate. Issues with a security impact should be treated specially - see the security disclosure process.

If the issue seems unclear - you aren't sure which labels to apply - ask the requester to clarify matters for you.

Keep our user communication guidelines in mind at all times, and commit to keeping up the conversation until you have enough information to complete triage.

Consider whether the issue is still valid. Especially for older issues, a ~"type::bug" may have been fixed since it was reported, or a ~"type::feature" may have already been implemented.

Be sure to check cross-reference notes from other issues or merge requests, they are a great source of information!
For instance, by looking at a cross-referenced merge request, you could see a "Picked into 8-13-stable, will go into 8.13.6." which would mean that the issue is fixed since the version 8.13.6.

If the issue meets the requirements, it may be appropriate to make a scheduling request - use your judgement!

You're done! The issue has all appropriate labels, and may now be in the backlog, closed, awaiting scheduling, or awaiting feedback from the requestor. Pick another, if you've got the time.

Issue Triage Practices

We're enforcing some of the policies automatically in
triage-ops, using the
@gitlab-bot user.
For more information about the automated triage, please read the
Triage Operations

That said, we can't automate everything. In this section we'll describe some of
the practices we're doing manually.

Shared responsibility issues

From time to time you may encounter issues for which it is difficult to pick a group or stage that should be responsible. It is likely that these issues address what is called Shared Responsibility Functionality of the product.

The approach for these is to use a decentralized triage process. The triage is not centralized in a single report or list, and it does not fall to one individual or group to have the responsibility to review those issues. This helps with scaling our triage operations to address a large number of issues that may fall into this shared responsibility category on an ongoing basis rather than in a recurring scheduled event.

The goal is to empower leadership at the group level (i.e. Product Manager and/or Engineering Manager) to make decisions on who, when and how these issues should be addressed. Higher-level management individuals and groups act as a backup to address escalations and make decisions when competing priorities make it difficult to decide on a course of action.

Initial triage

If you are triaging one of these issues as a GitLab engineer or as a quality department

manager, or if you are the author of the issue, please make your best effort to assign a group label to the issue as soon as possible after creation. You don't have to get it perfect, but just make a conscious effort to identify the group that is the best one set up for success to work on the issue.

You can ask yourself these questions when picking a group:

- Does the group directly lists this area in their product categories?
- Does the group works with the underlying technologies of the issue at hand?
- Have they done similar work in the past?
- Are they a foundational group that regularly engages in similar cross-cutting features?
- Can you identify the affected file and use git blame to see who was the author of the last change?
- Can you identify the Code Owners of the file?

To help with initially narrowing down the list of possible groups, you may review the Product Categories page or the Stage Groups Ownership Index page.

In any case, you should attempt to understand the nature of the issue by asking follow-up questions to the author if necessary, and then map the requirements to the group that best matches the skills or expertise required.

Secondary triage

Secondary triage happens when the issue has already been assigned to a group and now someone within the group (typically the PM or EM) is assessing the issue for prioritization and/or estimation. If you are the one doing this triage you may take one of the following courses of action:

1. If you determine that it falls squarely within your group's categories, then follow your own internal triage procedure.
1. If you think it falls squarely into another group's product categories, then ping the EM or PM of that other group async in a comment thread on the issue. Explain your reasoning as to why you think it falls under their purview and ask them to take ownership. Please ask and let them be the one to update the label when they have seen your comment and agreed that they are better suited to take on the issue.
1. If you think the issue falls into the shared responsibility category, first consider carefully having your team be the one contributing to the issue and owning it to see it through to completion. However, if you think your team does not have the skill or can not ramp up the necessary knowledge and skills to contribute to the issue in a timely manner, then try to identify other groups who might be better suited (or with whom your group can collaborate to deliver the functionality together). At that point, engage with them in a conversation and coordinate further triage of the issue until it has a clear owner. Alternatively, through this triage process you may collectively decide that the issue is not worth pursuing (e.g. perhaps considering its value vs effort). In this case, make sure to close the issue while providing a clear rationale for the decision.

As you work through the triage, exercise your judgement to decide when it is time to escalate issues to a higher level (i.e. senior management, directors or above) if you and your EM/PM peers can't agree on the value, severity, priority or group purview of the issue. For now, the

method to escalate is flexible and you can choose the right communication channel and modality for the situation.

As the DRI you should consider take additional steps to ensure the continued support of the affected area. This may involve putting forward proposals for the creation of new platform groups that can take the ongoing responsibility and technical strategy for the components in question. This of course does not preclude the need to take immediate action on the issue assigned to your group.

If as a result of the triage process a group is identified as qualified and willing to take ownership on a permanent basis, product and engineering leaders should officially document the type of ownership model and the team in the shared services components section of the Development handbook. Multiple groups may permanently share ownership of the same component if deemed appropriate.

It is important to keep in mind that throughout this process, as a leader in your group, you are deemed the initial Directly Responsible Individual (DRI) until the issue is resolved or someone else agrees to take over. Simply removing your group label without further triage conversations with other groups is not an acceptable or helpful action to take in this process. This aligns with our value of Results: global optimization.

Outdated issues

For issues that haven't been updated in the last 3 months the "Awaiting Feedback" label should be added to the issue. After 14 days, if no response has been made by anyone on the issue, the issue should be closed. This is a slightly modified version of the Rails Core policy on outdated issues.

If they respond at any point in the future, the issue can be considered for reopening. If we can't confirm an issue still exists in recent versions of GitLab, we're just adding noise to the issue tracker.

Duplicates

To find duplicates:

- Open the issue tracker for the project the issue is for.
- Enter relevant keywords from the issue.
- Scan through the first page of the result list.
- Check both open and closed issues.

Use the issue with the better title, description, or more comments and positive reactions as the canonical version. If you can't decide, keep the earlier issue.

Support issue message

If the issue is really a support request for help, you can use the Issue triage - support question comment template.

Duplicate issue message

If you find duplicates, you can post this message:

md
Hey {{author}}! Thanks for submitting this issue. It looks like a duplicate of {{issue}}. I'm marking your issue as a duplicate and close it.
Please add your thoughts and feedback on {{issue}}. Don't forget to upvote feature proposals.

/duplicate {{issue}}

Don't make any forward looking statements around milestone targets that the duplicate issue may be assigned.

Lean toward closing

We simply can't satisfy everyone. We need to balance pleasing users as much as possible with keeping the project maintainable.

- If the issue is a bug report without reproduction steps or version information, close the issue and ask the reporter to provide more information.
- If we're definitely not going to add a feature/change, say so and close the issue.

Label issues as they come in

When an issue comes in, it should be triaged and labeled. Issues without labels are harder to find and often get lost.

Be careful with severity labels. Underestimating severity can make a problem worse by suggesting resolution can wait longer than it should. Review available
Severity labels. If you are not certain of the right label for a bug, it is OK to overestimate the severity. But do get confirmation from a domain expert.

Take ownership of issues you've opened

Sort by "Author: your username" and close any issues which you know have been fixed or have become irrelevant for other reasons. Label them if they're not labeled already.

Product feedback issues

Some issues may not fall into the type labels, but they contain useful feedback on how GitLab features are used.
These issues should be mentioned to the product manager and labeled as ~"Product Feedback" in addition to the group, category and stage labels.
<https://gitlab.com/gitlab-org/gitlab/-/issues/324770> is an example of a Product Feedback issue.

Questions/support issues

If it's a question, or something vague that can't be addressed by the development team for whatever reason, close it and direct them to the relevant support

resources we have (e.g. <https://about.gitlab.com/get-help/>, our Discourse forum or emailing Support).

New labels

If you notice a common pattern amongst various issues (e.g. a new feature that doesn't have a dedicated label yet), suggest adding a new label in Slack or a new issue.

Reproducing issues

If possible, ask the reporter to reproduce the issue in a public project on GitLab.com. You can also try to do so yourself in the issue-reproduce group. You can ask any owner of that group for access.

Notes

The original issue about these policies is 17693. We'll be working to improve the situation from within GitLab itself as time goes on.

The following projects, resources, and blog posts were very helpful in crafting these policies:

- CodeTriage
- How to be an open source gardener
- Managing the Deluge of Atom Issues
- Handling Large OSS Projects Defensively
- My condolences, you're now the maintainer of a popular open source project
- The Art of Closing

SECTION: engineering/infrastructure/engineering-productivity/merge-request-triage.md

At GitLab, our mission is to change all creative work from read-only to read-write so that everyone can contribute. GitLab highly values community contribution and we want to continue growing community code contribution. GitLab encourages the community to file issues and open merge requests for our projects under the gitlab-org group, and for the gitlab-com/www-gitlab-com project. Their contributions are valuable, and we should handle them as effectively as possible. A central part of this is triage - the process of categorization according to type and product group.

Any GitLab team-member can triage merge requests. Keeping the number of un-triaged merge requests low is essential for maintainability, and is our collective responsibility. Consider triaging a few merge requests around your other responsibilities, or scheduling some time for it on a regular basis.

Triaging incoming wider community merge requests is divided between several departments. Quality Department maintains triage automation, Merge Request Coaches take on a partial merge request triage, and finally triage automation helps completing the triage process. Additionally, Contributor Success drives the community collaboration efforts and works with our

community to ensure they receive support and recognition for contributing to GitLab.

Merge request triage for the gitlab-org group

Triage levels (gitlab-org)

We define three levels of triage.

Initial triage (gitlab-org)

A merge request is considered initially triaged when it has a:

- ~"Community contribution" label applied
- "Thank you" message posted by @gitlab-bot with more details on the process

The initial triage is automated by the Contributor Success team via the Community contribution thank you note reactive triage automation.

Partial triage (gitlab-org)

A merge request is considered partially triaged when it has a:

- type label applied.
  - (For ~"type::bug" and ~"Deferred UX") It has a severity label applied.
- stage label applied.
- group label applied (e.g. ~"group:editor"). If no group label exists, the stage label is enough.

The partial triage is completed by Merge Request Coaches via the Newly created community merge requests triage report.

For MRs related to issues, partial triage can be completed by using the following quick action and confirming proper metadata:

shell
/copymetadata <issue link>

Complete triage (gitlab-org)

The complete Triage is divided into 3 subcategories depending on the community merge request state.

Complete triage for open merge requests (gitlab-org)

A merge request is considered completely triaged when it has:

- the workflow::ready for review label
- a reviewer assigned

Complete triage for merged merge requests (gitlab-org)

A merge request is considered completely triaged when it has a:

- milestone set if the merge request with the ~"Community contribution" label is merged.

This triage process is automated by the Contributor Success team via the Add milestone to community contributions on Triage Operations scheduled triage automation.

Inactive merge requests triage policy (gitlab-org)

The inactive merge request policy was created to enable GitLab teams to focus efforts on active merge request and prevent old merge requests from degrading into a state of disrepair. This is done by creating two thresholds at which GitLab team members attempt to move the merge request forward.

Contributor Success team members attempt to move merge requests that have reached the first threshold forward via the Community merge requests requiring attention triage report.

If that's not successful an Engineering Manager makes a decision at the second threshold. We value your effort - that's why all decisions to close a merge request are made by a human, and are not automated.

Wider community merge request triage SLOs (gitlab-org)

Community contributions are valuable, and we should handle them as effectively as possible to ensure swift feedback to community and increase engagement. To achieve that we define the following Service-level objectives (SLOs):

Triage Level or Response Metric	SLO
Initial Triage	2 hours (this is automated)
Partial Triage	7 days
Complete Triage for Merged Merge Requests	1 day (this is automated for gitlab-org/gitlab)
Time to assign reviewer	2 hours (this is automated)

If an SLO isn't met, reach out to the Contributor Success team.

Merge request triage for the gitlab-com/www-gitlab-com project

Triage levels (gitlab-com/www-gitlab-com)

The GitLab Website is owned and managed by a different team than GitLab.org; thus, a further triage process must be defined.

Initial triage (gitlab-com/www-gitlab-com)

Same as for the gitlab-org group above.

Complete triage (gitlab-com/www-gitlab-com)

The Complete Triage is divided into 3 subcategories depending on the community merge request state.

Complete triage for open merge requests (gitlab-com/www-gitlab-com)

A merge request is considered completely triaged when it has:

- a reviewer assigned by a member of the GitLab Website Community Team.
- been reviewed by a reviewer.

Typically, the reviewer is the code owner of the page the merge request is updated. If there is no code owner assigned, the triager will reach out to the relevant team the page belongs to identify a reviewer.

Complete triage for idle merge requests (gitlab-com/www-gitlab-com)

A merge request is considered completely triaged when it:

- is closed following the closing policy for merge requests.

This triage process is being done manually on a case-by-case basis by a member of the GitLab Website Community Team or the relevant code owner.

Wider community merge request triage SLOs (gitlab-com/www-gitlab-com)

Community contributions are valuable, and we should handle them as effectively as possible to ensure swift feedback to community and increase engagement. To achieve that we define the following Service-level objectives (SLOs):

Triage Level	Triage SLO
Initial triage	2 hours (this is automated)
Time to assign reviewer	7 days (this is automated)
Complete triage for idle merge requests	7 days

If an SLO isn't met, reach out to @gitlab-com-community in the merge request.

SECTION: engineering/infrastructure/engineering-productivity/project-management.md

Work prioritization

The Engineering Productivity team has diverse responsibilities and reactive work. Work is categorized as planned and reactive.

Guiding principles

- We focus on OKRs, corrective actions and preventative work.
- We adhere to the general release milestones like %x.y.

- We are ambitious with our targeted planned work per milestone. These targets are not reflective of a commitment. Reactive work load will ebb and flow and we do not expect to accomplish everything planned for the current milestone.
- Priority labels are used to indicate relative priority for a milestone.

Weighting

We follow the department weighting guidelines to relatively weight issues over time to understand a milestone velocity and increase predictability.

When weighting, think about knowns and complexity related to recently completed work. The goal with weighting is to allow for some estimation ambiguity that allows for a consistent predictable flow of work each milestone.

Prioritization activities

When	Activity	DRI
Weekly	Assign ~priority::1, ~priority::2 issues to a milestone	Engineering Productivity Engineering Manager
Weekly	Weight issues identified with ~"needs weight"	Engineering Productivity Backend Engineer
Weekly	Prioritize all ~"Engineering Productivity" issues	Engineering Productivity Engineering Manager
2 weeks prior to milestone start	Milestone planned work is identified and scheduled	Engineering Productivity Engineering Manager
2 weeks prior to milestone start	Provide feedback on planned work	Engineering Productivity team
1 week prior to milestone start	Transition any work that is not in progress for current milestone to upcoming milestone	Engineering Productivity Engineering Manager
1 week prior to milestone start	Adjust planned work for upcoming milestone	Engineering Productivity Engineering Manager
1 week prior to milestone start	Final adjustments to planned scope	Engineering Productivity team
During milestone	Adjust priorities and scope based on newly identified issues and reactive workload	Engineering Productivity Engineering Manager

Projects

The Engineering productivity team currently works cross-functionally and our task ownership spans multiple projects.

The list below is ordered based on aligned priorities and includes primary domain experts for communication as well as a documentation reference for self-service.

Project	Domain Knowledge	Documentation
GitLab CI Pipeline configuration optimization and stability	Jen-Shin, David, Jenn	Pipelines for the GitLab project
Triaging master-broken	Jenn, Nao	Broken Master

GitLab Development Kit (GDK) continued development	Nao, Peter	GitLab Development Kit
Triage operations for issues, merge requests, community contributions	Jenn, Alina	triage-ops
Review Apps	David, Rémy	Using review apps in the development of GitLab
Triage engine, used by GitLab triage operations	Jen-Shin, Rémy	GitLab Triage
Danger & Dangerfiles (includes Reviewer roulette) for shared Danger rules and plugins	Rémy, Jen-Shin, Peter	gitLab-dangerfiles Ruby gem for shared Danger rules and plugins
JiHu	Jen-Shin	JiHu Support
Development department metrics for measurements of Quality and Productivity	Jenn, Rémy	Development Department Performance Indicators
RSpec Profiling Statistics for profiling information on RSpec tests in CI	Peter	rspecprofilingstats
RuboCop & shared RuboCop cops	Peter	gitLab-styles Ruby gem for shared RuboCop cops
Feature flag alert for reporting on GitLab feature flags	Rémy	GitLab feature flag alert
Chatops (especially for feature flags toggling)	Rémy	Chatops scripts for managing GitLab.com from Slack
CI/CD variables, Triage ops, and Internal workspaces infrastructure	David, Rémy	Engineering Productivity infrastructure
Tokens management	Rémy	"Rotating credentials" runbook
Gems management	Rémy	Rubygems committee project
Shared CI/CD config & components	David, Rémy	gitlab-org/quality/pipeline-common and gitlab-org/components
Dependency management (Gems, Ruby, Vue, etc.)	Jen-Shin, Peter	Renovate GitLab bot
Quality toolbox	David, Rémy	Quality toolbox

Issues

Issues currently worked on

Our team's Quality: Engineering Productivity board shows the current ownership of workload / issues maintained by team members in Engineering Productivity team.

Asynchronous issue updates

Communicating progress is important but status doesn't belong in one on ones as it can be more appropriately communicated with a broader audience using other methods. The "standup" model used by a lot of organizations practicing scrum assumes a certain time of day for those to happen. In the context of a timezone distributed team, there is no "9am" that the team shares. Additionally, the act of losing and gaining context after completing work for the day only to gain it again to share a status update is context switching. The intended audience of the standup model assumes that it's just the team but in GitLab's model, that means folks need to be aware of where this is being communicated (slack, issues, other). Since this information isn't available to the intended audience, the information needs to be duplicated which at worst means there's no single source of truth and at a minimum means copy pasting information.

The proposal is to trial using an Asynchronous Issue Update model, similar to what the Package Group uses. This process would replace the existing daily standup update we post in Slack with Geekbot. The time period for the trial would be a milestone or two, depending on feedback cycles.

The async daily update communicates the progress and confidence using an issue comment and the milestone health status using the Health Status field in the issue. A daily update may be skipped if there was no progress. Merge requests that do not have a related issue should be updated directly. It's preferable to update the issue rather than the related merge requests, as those do not provide a view of the overall progress. Where there are blockers or you need support, Slack is the preferred space to ask for that. Being blocked or needing support are more urgent than email notifications allow.

When communicating the health status, the options are:

- on track - when the issue is progressing as planned
- needs attention - when the issue requires attention or intervention to keep it on schedule
- at risk - when there is a risk the issue will not be completed according to schedule

The async update comment should include:

- what percentage complete the work is, in other words, how much work is done to put all the required MRs in review
- the confidence of the person that their estimate is correct
- notes on what was done and/or if review has started
- it could be good to specify the relevant dependencies in the update, if there are multiple people working on it

Example:

markdown
Status: 20% complete, 75% confident

Expecting to go into review tomorrow.

Include one entry for each associated MR

Example:

markdown
Issue status: 20% complete, 75% confident

Expecting to go into review tomorrow.

MR statuses:

- !11111+ - 80% complete, 99% confident - docs update - need to add one more section
- !21212+ - 10% complete, 70% confident - api update - database migrations created, working on creating the rest of the functionality next

How to measure confidence?

Ask yourself, how confident am I that my % of completeness is correct?.

For things like bugs or issues with many unknowns, the confidence can help communicate the level of unknowns. For example, if you start a bug with a lot of unknowns on the first day of the milestone you might have low confidence that you understand what your level of progress is. Your confidence in the work may go down for whatever reason, it's acceptable to downgrade your confidence. Consideration should be given to retrospecting on why that happened.

Epics

Weekly epic updates

A weekly update should be added to epics you're assigned to and/or are actively working on. The update should provide an overview of the progress across the feature. Consider adding an update if epic is blocked, if there are unexpected competing priorities, and even when not in progress, what is the confidence level to deliver by the expected delivery date. A weekly update may then be skipped until the situation changes. Anyone working on issues assigned to an epic can post weekly updates.

The epic updates communicate a high level view of progress and status for quarterly goals using an epic comment. It does not need to have issue or MR level granularity because that is part of each issue updates.

The weekly update comment should include:

- Status: ok, so-so, bad? Is there something blocked in the general effort?
- How much of the total work is done? How much is remaining? Do we have an ETA?
- What's your confidence level on the completion percentage?
- What is next?
- Is there something that needs help/support? (tag specific individuals so they know ahead of time)

Examples

Some good examples of epic updates that cover the above aspects:

- <https://gitlab.com/groups/gitlab-org/-/epics/8628note1090732793>
- <https://gitlab.com/groups/gitlab-org/-/epics/5152note1029337901>

Reviewers and maintainers

Upon joining the Engineering productivity team, team members are granted either developer, maintainer, or owner access to a variety of core projects. For projects where only developer access is initially granted, there are some criteria that should be met before maintainer access is granted.

- GitLab Tooling and Pipeline configuration
  - GitLab Tooling and Pipeline configuration consists of scripts and config files used for both local development and for CI pipelines. Changes made to these files have wide impact to developer experience at GitLab.
    - Please note: despite being two different code categories, the Reviewer roulette is designed to suggest @gl-quality/tooling-maintainers to review both Tooling and Pipeline

configuration MRs. We have an issue to split up maintainers for GitLab Tooling and GitLab Pipeline configuration into @gl-quality/tooling-maintainers and @gl-quality/pipeline-maintainers. For now, everyone in @gl-quality/tooling-maintainers is required to have the knowledge to review both code changes.
  - To become a Tooling and Pipeline configuration maintainer, one must have:
    - Read https://docs.gitlab.com/ee/development/pipelines/index.html and https://docs.gitlab.com/ee/development/pipelines/internals.html and is familiar with GitLab's internal pipeline configuration rules and patterns.
    - Authored 5 merged MRs for Tooling maintenance and improvements.
    - Authored 5 merged MRs for Pipeline configuration maintenance and improvements.
    - Reviewed 10 MRs demonstrate good understanding of tooling and GitLab pipeline configurations.
    - After completing the above requirements, a merge request is created in the handbook to update their team member YAML outlining the reasons why they should be a maintainer and list all 20 merge requests to help aid with review. This MR must be approved by a member of @gl-quality/tooling-maintainers.
- GitLab Triage
  - Authored 5 merged MRs.
  - Reviewed 5 MRs.
  - After completing the above requirement the maintainer should be vetted by an existing maintainer in the Engineering Productivity team. An issue should be created in the project outlining the reasons why this person should be a maintainer. List all 10 MRs in the issue to help aid with review.
  - After the issue has been reviewed and approved by manager of the Engineering Productivity team, an access request will be created to grant the engineer maintainer role.
- GitLab Roulette
  - Authored or reviewed 2 MRs in total.
- GitLab Development Kit
  - In general, we expect that team members will generally feel comfortable and will be granted maintainer access once they have:
    - Authored 5 MRs related to new features or improvements for GDK.
    - Reviewed 10 MRs.
  - After completing the above requirement, the maintainer should be vetted by an existing maintainer in GDK. A merge request should be created in the www-gitlab-com repository outlining the reasons why this person should be a maintainer.
- GitLab Styles
  - Authored or reviewed 2 MRs in total.
- GitLab Dangerfiles
  - Authored or reviewed 5 MRs in total.
- GitLab Quality Test Tooling
  - Authored or reviewed 5 MRs in total.
- Triage Ops
  - Authored or reviewed 10 MRs in total.

Becoming a maintainer

The following guidelines will help you to become a maintainer. Remember that there is no specific
timeline on this, and that you should work together with your manager and current maintainers.

To start the process of becoming a maintainer, see the maintainer section
of the code review guidelines.

In general, you're required to author and review 3 - 10 MRs that demonstrate good overall understanding
of the existing codebase and framework. See the section above for further details of the requirements.
You can seek out more opportunities to work on framework improvements by asking on the quality Slack channel.

Your reviews should aim to cover maintainer responsibilities as well as reviewer responsibilities.
Your approval means you think it is ready to merge.

It is your responsibility to set up any necessary meetings to discuss your
progress with current maintainers, as well as your manager. These can be at any
frequency that is right for you.

SECTION: engineering/infrastructure/engineering-productivity/test-intelligence.md

Introduction

As the owner of pipeline configuration for the GitLab project, the Engineering Productivity
team has adopted several test intelligence strategies aimed to improve pipeline efficiency with
the following benefits:

- Shortened feedback loop by prioritizing tests that are most likely to fail
- Faster pipelines to scale better when Merge Train is enabled

These strategies include:

- Predictive test jobs via test mapping
- Fail-fast job
- Re-run previously failed tests early
- Selective jobs via pipeline rules
- Selective jobs via labels

Predictive test jobs via test mapping

Tests that provide coverage to the code changes in each merge request are most likely to fail.
As a result, merge request pipelines for the GitLab project run only the predictive set of
tests by default. These include:

- RSpec predictive jobs which runs relevant RSpec tests that are mapped to the code changes
- Jest predictive jobs which runs relevant Jest tests that are mapped to the code changes

See <https://docs.gitlab.com/ee/development/pipelines/index.htmlpredictive-test-jobs-before-a-merge-request-is-approved> for more information.

Fail-fast job

There is a fail-fast job in each merge request pipeline aimed to run all the RSpec tests that provide coverage for the code changes, hence are most likely to fail. It uses the same testfilefinder gem for test mapping. The job provides faster feedback by running early and stops the rest of the pipeline right away if any of the fail-fast job tests fail.
Take a look at this youtube video for details on how GitLab implements the fail-fast job with testfilefinder.
Note that the current design only works with low-impacting merge requests which are only mapped to a small set of tests. If there is a large number of tests that are likely to fail for a merge request, putting them in a single job is not feasible and could result in a long-running bottleneck which defeats its purpose.

See <https://docs.gitlab.com/ee/development/pipelines/index.htmlfail-fast-job-in-merge-request-pipelines> for more information.

Premium GitLab customers, who wish to incorporate the Fail-Fast job into their Ruby projects, can set it up with our Verify/Failfast template.

Re-run previously failed tests early

Tests that previously failed in a merge request are likely to fail again, so they provide the most urgent feedback in the next run.
To grant these tests the highest priority, the GitLab pipeline prioritizes previously failed tests by re-running them early in a dedicated job, so it will be one of the first jobs to fail if attention is needed.

See <https://docs.gitlab.com/ee/development/pipelines/index.htmlre-run-previously-failed-tests-in-merge-request-pipelines> for more information.

Selective jobs via pipeline rules

The GitLab pipeline consists of hundreds of jobs, but not all are necessary for each merge request. For example, a merge request with only changes to documentation files do not need to run any backend tests, so we can exclude all backend test jobs from the pipeline.
See specify-when-jobs-run-with-rules for how to include/exclude CI jobs based on file changes.
Most of the pipeline rules for the GitLab project can be found in <https://gitlab.com/gitla